

2.5 PUT, PATCH and DELETE Mappings in Spring Web MVC

Note: In *Spring Data JPA*, repository methods like `findById(Long id)` return an `Optional<T>` by design.

What is `Optional<T>` in Java?

- `Optional<T>` is a `container` object that may or may not contain a *non-null value*. It was introduced in *Java 8* to avoid `NullPointerException` (NPE) by making null-handling explicit.
- Think of it like a box:
 - If the box has something inside → value is *present*.
 - If the box is empty → value is *absent*.

@PutMapping :

- `@PutMapping` is a specialized annotation in the *Spring Web MVC framework* used to map HTTP *PUT* requests to specific handler methods in a *controller class*.
- It is part of the *Spring Boot RESTful web service support* and is typically used when *updating* an *existing resource* in the application.

Purpose

- To handle HTTP *PUT* requests.
- Commonly used for *replacing entire resources* (as per REST standards).

Ex.

```
@PutMapping("/{id}")
public EmployeeDTO replaceEmployeeById(@PathVariable("id") Long id,
                                       @RequestBody EmployeeDTO updatedEmployee) { // controller layer snippet
    return employeeService.replaceEmployeeById(id, updatedEmployee);
}

public EmployeeDTO replaceEmployeeById(Long id, EmployeeDTO updatedEmployee) { // service layer snippet
    updatedEmployee.setId(id);
    EmployeeEntity existingEmployee = repository.findById(id).orElseThrow(() -> new RuntimeException("No such employee exist"));
    mapper.map(updatedEmployee, existingEmployee);
    repository.save(existingEmployee);
    return updatedEmployee;
}
```

@DeleteMapping :

- `@DeleteMapping` in Spring Boot is an annotation used in *Spring MVC* to handle *HTTP DELETE* requests.

Purpose

- It is typically used in *REST APIs* to *delete* a *resource* (e.g., a database record) identified by a specific *ID* or criteria.

Key Points:

- Works like `@GetMapping`, `@PostMapping`, and `@PutMapping` but for *DELETE* requests.
- Mapped to a *controller method* that performs the *delete* operation.
- Often combined with `@PathVariable` or `@RequestParam` to *identify which resource to delete*.

Ex.

```
@DeleteMapping("/{id}")
public Boolean deleteEmployee(@PathVariable("id") Long id) { // controller layer snippet
    return employeeService.deleteEmployeeById(id);
}

public Boolean deleteEmployeeById(Long id) { // service layer snippet
    if (repository.existsById(id)) {
        repository.deleteById(id);
        return true;
    }
    return false;
}
```

When the client sends an HTTP DELETE request to `/employees/1`, the `deleteEmployee` method will be called, which removes the employee with ID 1.

@Patching:

- **@Patching** in Spring Boot is used to handle **HTTP PATCH** requests, which are meant for **partial updates** of a resource.
- PUT usually **replaces the entire resource** with what you send.
- PATCH only changes **specific fields** in the resource **without touching the others**.

When to use:

- If you want **only certain fields** to be updated.
- Useful for **user profile updates** (e.g., changing just the phone number).

Ex.

```
@Patching("/{id}")
public EmployeeDTO updateEmployeeById(@PathVariable("id") Long id,
                                     @RequestBody Map<String, Object> updates) { // controller layer snippet
    return employeeService.updateEmployeePartially(id, updates);
}

public EmployeeDTO updateEmployeePartially(Long id, Map<String, Object> updates) { // service layer snippet
    if (!existsById(id)) return null;
    EmployeeEntity existingEmployee = repository.findById(id).get();
    updates.forEach((field, value) -> {
        Field fieldToBeUpdate = ReflectionUtils.findField(EmployeeEntity.class, field);
        if (fieldToBeUpdate != null) {
            fieldToBeUpdate.setAccessible(true);
            ReflectionUtils.setField(fieldToBeUpdate, existingEmployee, value);
        }
    });
    return mapper.map(repository.save(existingEmployee), EmployeeDTO.class);
}
```

What is **Reflection** in Java?

- Reflection lets you **inspect** and **modify** classes, methods, and fields at **runtime**, even if you **don't know** them at **compile time**.
- It's like saying:
"I don't know exactly what this object looks like in code, but I'll peek inside it and **change something** while the program is running."

Why **ReflectionUtils** here?

- In the **@Patching** example, we don't know ahead of time **which fields** the user wants to update. The request body might be:

```
{
  "name" : "xyz"
  "age" : "23"
}
```
- We get this as a **Map<String, Object>** → each key is a **field name** in the **EmployeeEntity**. We then use:
 - **ReflectionUtils.findField()** → find a **field** in a class by its **name** only (e.g. age, name)
 - **ReflectionUtils.setField()** → set a **value** to that field dynamically, even if it's **private**.

Why not just call setters?

Because:

1. We don't know **which setters** to call ahead of time — it's decided by the **request data**.
2. Writing **if statements** for every possible field is repetitive.
3. **Reflection** makes it **generic** — works for any field without **hardcoding**.

Here, **ReflectionUtils** is used so we can **dynamically update only the fields present in the PATCH request**, without knowing them at **compile time**.

What is **ResponseEntity** ?

- In Spring Boot, **ResponseEntity<T>** is a class that represents the **entire HTTP response**.
- It lets you control:
 - **HTTP Status Code** (e.g., 200 OK, 404 NOT_FOUND)
 - **Headers** (e.g., custom Content-Type, Authorization)
 - **Body** (**actual data** you send back — JSON, String, DTO, etc.)

In short: **ResponseEntity** = **Body** + **Status Code** + **Headers** in one object.

When to Use ResponseEntity :

You use it when you want **more control** over the **HTTP response** rather than just returning data.

1. You need to send different **status codes** based on conditions.

```
if (employee == null) return ResponseEntity.notFound().build();  
return ResponseEntity.ok(employee);
```
2. You want to set **custom HTTP headers**.
3. You want to send **empty responses** (no body) with a specific status.
4. You need to send an **error message** with a different status code (e.g., **400** Bad Request).

@GetMapping

```
@GetMapping  
public ResponseEntity<List<EmployeeDTO>> getAllEmployees() {  
    List<EmployeeDTO> employeeList = employeeService.getAllEmployees();  
  
    return (employeeList.isEmpty()) ?  
        ResponseEntity.noContent().build() : ResponseEntity.ok(employeeList);  
}
```

employeeService.getEmployeeById(id): This is calling your service layer method to fetch the employee of **Optional<EmployeeDTO>** type.

.map(ResponseEntity::ok): This is **Optional's map()** method — not **ModelMapper's**. It says: "If the **Optional** contains a value, **take it** and give it to **ResponseEntity.ok()** to create a **200 OK** response with that value in the body."

If the **Optional** is empty, **.map()** will **not** run at all — it just keeps it empty.

.orElse(ResponseEntity.notFound().build()): If the **Optional** is **empty** (employee not found): This executes **ResponseEntity.notFound().build()** → Which returns a **404 Not Found** response with **no** body.

@PostMapping

```
@PostMapping  
public ResponseEntity<EmployeeDTO> createNewEmployee(@RequestBody EmployeeDTO employee) {  
    EmployeeDTO employeeDTO = employeeService.createNewEmployee(employee);  
    return new ResponseEntity<>(employeeDTO, HttpStatus.CREATED);  
}
```

It returns new **ResponseEntity<>()** with status code **created 201**.

@PutMapping

```
@PutMapping("/{id}")  
public ResponseEntity<EmployeeDTO> replaceEmployeeById(@PathVariable("id") Long id,  
    @RequestBody EmployeeDTO updatedEmployee) {  
    return employeeService.replaceEmployeeById(id, updatedEmployee)  
        .map(ResponseEntity::ok)  
        .orElse(ResponseEntity.notFound().build());  
}
```

It returns **body** with **200 ok** if available otherwise return **HttpStatus 204 no content**.